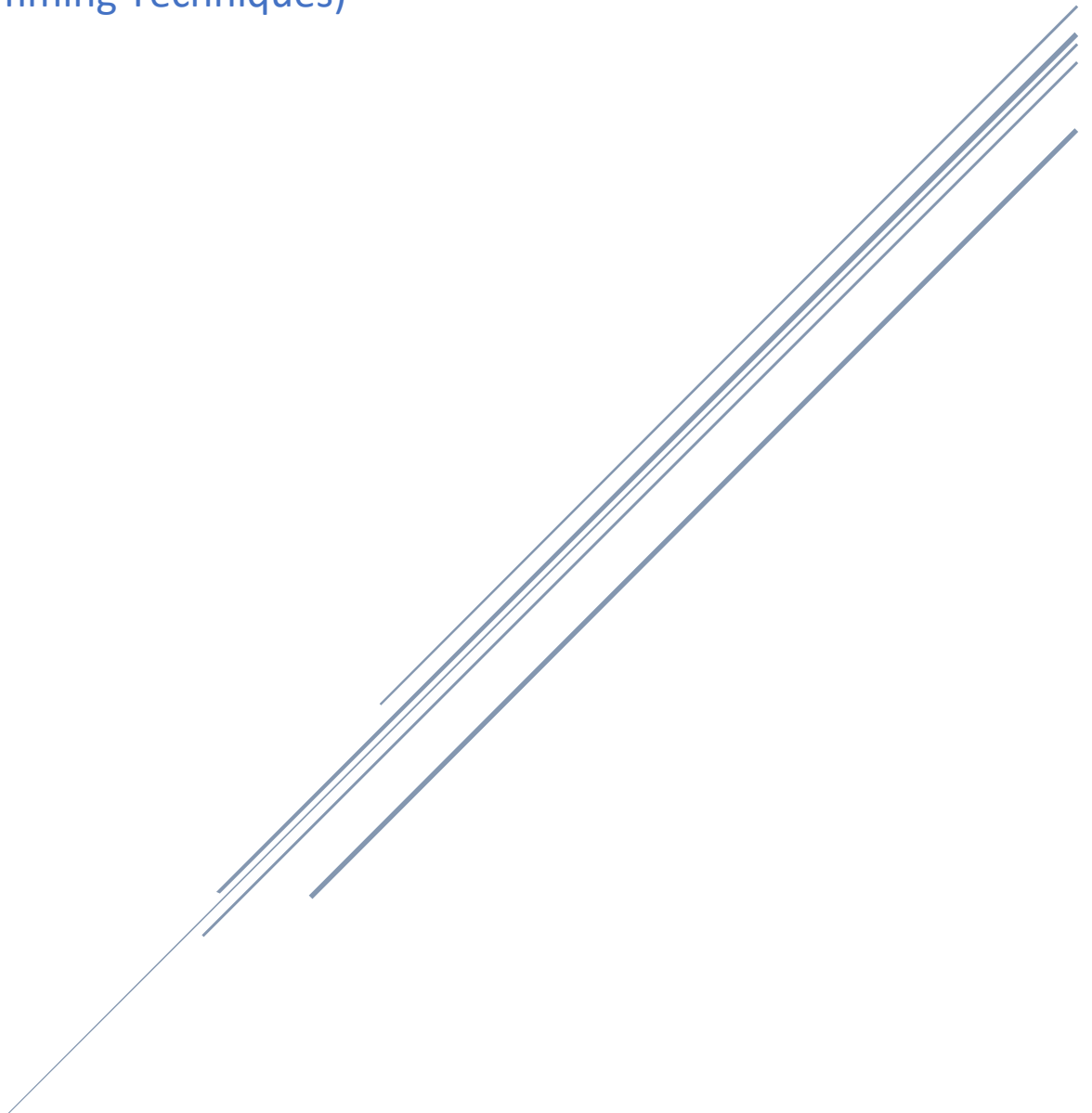


AI-ASSISTED PROGRAMMING IN MODERN SOFTWARE DEVELOPMENT

Student Management System (Assessment 2: Advanced
Programming Techniques)



S.A.T.Gimantha
17335

➤ Declaration

I hereby declare that this assignment is my own work. The project was developed using both traditional programming techniques and AI-assisted programming tools in accordance with the assignment requirements. All external resources used have been properly acknowledged.

➤ Acknowledgement

I would like to express my sincere gratitude to the Lecturer-in-Charge of IT-3003 – Advanced Programming Techniques for the valuable guidance, knowledge, and support provided throughout this course. The concepts and practical skills taught during the lectures greatly contributed to my understanding of Object-Oriented Programming and helped me successfully complete this assignment.

I also acknowledge the use of AI-assisted programming tools during the development of this project. These tools were used to assist with software design, code generation, debugging, UML diagram creation, and report preparation. All AI-generated content was carefully reviewed, modified, tested, and understood before being incorporated into the final implementation.

Completing this assignment has strengthened my understanding of Java programming and provided valuable practical experience in applying Object-Oriented Programming concepts to develop a simple Student Management System.

➤ **Table of Contents**

1.Introduction

2.Task 1

2.1 What is AI-Assisted Programming?

2.2 Types of AI Coding Tools

2.3 Advantages

2.4 Risks and Limitations

3.Task 2

3.1 Traditional Approach Solution

3.2 AI-Assisted Approach Solution

3.3 Comparison

3.4 AI-Prompts Used and UML Diagrams

4.Task 3

Critical Analysis

5.Task 4

Conclusion

6.References

7.GitHub Repository

1.Introduction

Artificial Intelligence (AI) has become an important technology in modern software development. AI-assisted programming tools are transforming how software engineers write, debug, optimize, and maintain source code. These tools provide intelligent suggestions, automatically generate code, identify programming errors, and improve overall development productivity.

Traditional programming requires developers to manually design algorithms, implement software components, test functionality, and identify bugs. Although this approach provides a deep understanding of programming concepts, it often requires considerable time and effort.

With recent advancements in Artificial Intelligence, programming assistants such as ChatGPT, GitHub Copilot, Amazon CodeWhisperer, and other AI-based development tools have significantly improved software development workflows. These tools help developers produce high-quality code more efficiently while reducing repetitive programming tasks.

This project investigates the impact of AI-assisted programming by developing a Student Management System using Java Object-Oriented Programming principles. Two development approaches are considered: a traditional programming approach and an AI-assisted programming approach. The project evaluates both methods in terms of development time, code quality, readability, maintainability, and error handling. Furthermore, the ethical implications and limitations of AI-assisted programming are critically discussed.

2. Task 1 – Conceptual Understanding

2.1 What is AI-Assisted Programming?

Artificial Intelligence (AI)-Assisted Programming refers to the use of artificial intelligence technologies to support software developers during the software development process. AI-powered programming tools help developers generate source code, identify programming errors, explain algorithms, debug applications, and improve software quality. Rather than replacing programmers, these tools function as intelligent assistants that enhance productivity and simplify complex programming tasks.

Recent advancements in machine learning and natural language processing have made AI-assisted programming increasingly popular in modern software engineering. Developers can describe a programming requirement using natural language, and AI tools can generate code suggestions or even complete functions based on the given instructions. This reduces development time and allows programmers to focus on solving more complex problems.

AI-assisted programming is now widely used in both industry and education. It supports learning by explaining programming concepts, suggesting improvements, and helping developers understand unfamiliar programming languages and frameworks. However, developers must always review and test AI-generated code before integrating it into software projects to ensure correctness and reliability.

For this assignment, AI-assisted programming was used to assist in designing and implementing a Student Management System using Java. The generated code was carefully reviewed, modified where necessary, and tested before being included in the final solution.

2.2 Types of AI Coding Tools

Many AI-powered programming assistants are available today. These tools provide different features to support software development.

2.2.1 ChatGPT (OpenAI)

ChatGPT is an AI-powered conversational assistant capable of generating source code, explaining programming concepts, debugging errors, and assisting with software documentation.

Developers can interact with ChatGPT using natural language to receive coding assistance across multiple programming languages.

Main Features

- Generates source code
- Explains programming concepts
- Helps debug errors
- Assists with documentation
- Supports UML design discussions

2.2.2 GitHub Copilot

GitHub Copilot is an AI-based coding assistant developed by GitHub and OpenAI. It integrates directly with code editors such as Visual Studio Code and provides intelligent code completion while developers write programs.

Main Features

- Real-time code suggestions
- Function generation
- Auto-completion
- Context-aware programming support

2.2.3 Amazon CodeWhisperer

Amazon CodeWhisperer is an AI coding assistant developed by Amazon Web Services (AWS). It provides code recommendations and security suggestions, particularly for cloud application development.

Main Features

- Secure code recommendations
- AWS service integration
- Code generation
- Vulnerability detection

2.2.4 Google Gemini

Google Gemini is an AI assistant that supports programming, debugging, documentation, and problem solving. It can generate code snippets and explain programming logic in multiple programming languages.

Main Features

- Code generation
- Debugging assistance
- Documentation support
- Programming explanations

2.3 Advantages of AI-Assisted Programming

AI-assisted programming offers several advantages that improve software development efficiency and productivity.

2.3.1 Increased Productivity

AI tools automate repetitive coding tasks, allowing developers to focus on solving more complex programming problems. This significantly reduces software development time.

2.3.2 Faster Code Generation

Developers can describe programming requirements in natural language, and AI tools can quickly generate code templates or complete functions. This accelerates the implementation process.

2.3.3 Improved Code Quality

AI tools often recommend better programming practices and identify syntax errors before the program is executed. This helps developers produce cleaner and more maintainable code.

2.3.4 Learning Support

Students and beginner programmers can use AI assistants to understand programming concepts, algorithms, and object-oriented programming techniques. AI also provides explanations that make learning easier.

2.3.5 Faster Debugging

AI-assisted tools can identify programming errors, explain compiler messages, and recommend possible solutions, reducing the time spent debugging software.

2.3.6 Documentation Assistance

AI can automatically generate comments, documentation, and code explanations, making software documentation easier and improving code readability.

2.4 Risks and Limitations of AI-Assisted Programming

Despite its advantages, AI-assisted programming also has several limitations that developers should consider.

2.4.1 Incorrect Code Generation

AI-generated code is not always accurate. Developers must carefully review, modify, and test all generated code before using it in software applications.

2.4.2 Security Risks

AI tools may occasionally generate code containing security vulnerabilities or inefficient programming practices. Developers remain responsible for ensuring software security.

2.4.3 Reduced Learning

Excessive reliance on AI may reduce students' ability to think critically and solve programming problems independently. It is important to understand the generated code rather than simply copying it.

2.4.4 Limited Understanding of Context

AI tools may not fully understand the overall objectives or business requirements of a software project. Human judgment is still necessary when designing and implementing software systems.

2.4.5 Dependency on AI Tools

Over-dependence on AI assistants may reduce a programmer's confidence and ability to write code without external support. Developers should continue practising manual coding skills.

2.4.6 Ethical Considerations

AI-generated code should be used responsibly. Developers should acknowledge the use of AI tools when required and ensure that generated solutions comply with academic integrity and professional ethical standards.

3.Task 02

3.1Version-A (Traditional Approach)

```
PS C:\Users\THEVINDR\Documents\student management system> java -XStudentManagementSystem.java
Enter Student Name: thevin
Enter Student ID: 124
Enter Course: computer science

Enter Lecturer Name: kasun
Enter Lecturer ID: 101
Enter Subject: computer science

----- Student Management System -----

Role: Student
Role: Lecturer

Student Name: thevin
Student ID: 124
Course: computer science

Lecturer Name: kasun
Lecturer ID: 101
Subject: computer science
PS C:\Users\THEVINDR\Documents\student management system>
```

3.2 Version-B (AI-Assisted Approach)

```
Student added successfully.

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter Choice : 2
=====
Student ID : 101
Name      : thevindu
Email     : thevindugimantha@gmail.com
Phone     : 0740188622
Department : statistics
=====

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter Choice : 3
Enter Student ID : 101
=====
Student ID : 101
Name      : thevindu
Email     : thevindugimantha@gmail.com
Phone     : 0740188622
```

```
=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter Choice : 4
Student ID : 101
New Name : thevi
New Email : none
New Phone : none
New Department : statistics
Student updated successfully.

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter Choice : 2
=====
Student ID : 101
Name      : thevi
Email     : none
Phone     : none
Department : statistics
=====
```

```
=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter Choice : 3
Enter Student ID : 101
=====
Student ID : 101
Name      : thevindu
Email     : thevindugimantha@gmail.com
Phone     : 0740188622
Department : statistics
=====

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter Choice : 4
Student ID : 101
New Name : thevi
New Email : none
New Phone : none
New Department : statistics
Student updated successfully.
```

```
=====
STUDENT MANAGEMENT SYSTEM
=====
```

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

```
Enter Choice : 5
Student ID : 101
Student deleted successfully.
```

```
=====
STUDENT MANAGEMENT SYSTEM
=====
```

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

```
Enter Choice : 6
Thank You!
```

```
=====
STUDENT MANAGEMENT SYSTEM
=====
```

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

```
Enter Choice : 2
```

```
-----
Student ID : 101
Name       : thevindu
Email      : thevindugimantha@gmail.com
Phone      : 0740188622
Department : statistics
-----
```

```
=====
STUDENT MANAGEMENT SYSTEM
=====
```

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

```
Enter Choice : 3
Enter Student ID : 101
```

```
-----
Student ID : 101
Name       : thevindu
Email      : thevindugimantha@gmail.com
Phone      : 0740188622
Department : statistics
-----
```

3.3 Comparison

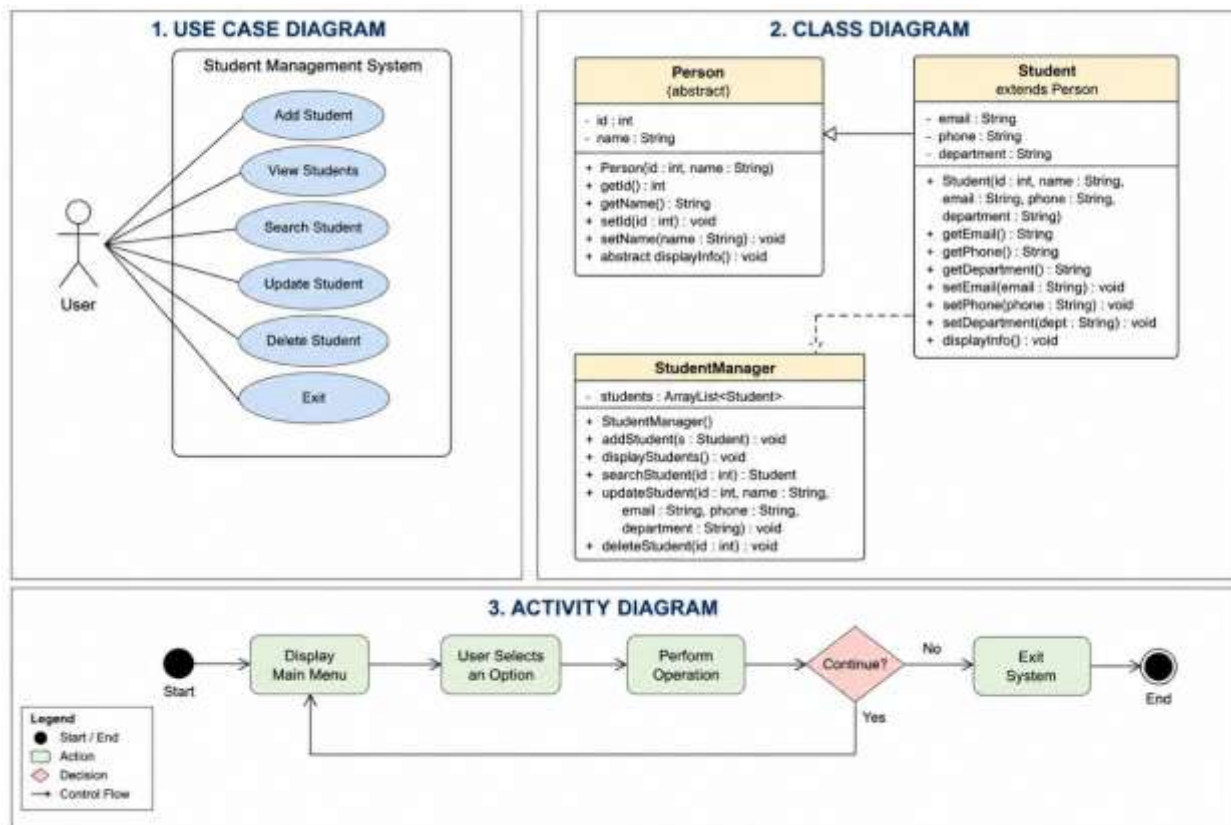
Criteria	Version A – Traditional Programming	Version B – AI-Assisted Programming
Development Time	The program was written manually from scratch, requiring more time for planning, coding, and debugging.	AI-assisted development reduced coding time by generating code suggestions and providing implementation guidance, allowing the system to be completed much faster.
Code Quality	The program correctly demonstrates OOP concepts but has limited functionality. It mainly displays one student and one lecturer entered by the user.	The program is more structured and organized. It includes separate classes for different responsibilities and supports complete CRUD operations (Add, View, Search, Update, Delete), resulting in a more practical Student Management System.
Error Handling	The implementation relies mainly on normal program execution. It does not validate duplicate student IDs or handle invalid menu selections extensively.	Basic validation is included, such as checking whether a student exists before updating or deleting records and preventing duplicate student IDs. Although custom exceptions were intentionally omitted for simplicity, the system handles common situations more effectively.
Readability and Maintainability	Since all classes are written in a single Java file, the program becomes more difficult to maintain as additional features are added. Modifying one class may affect the entire program.	The system is divided into separate Java files (Person, Student, StudentManager, and Main), making the code more modular, easier to read, and easier to maintain. Each class has a clear responsibility, improving overall software organization.

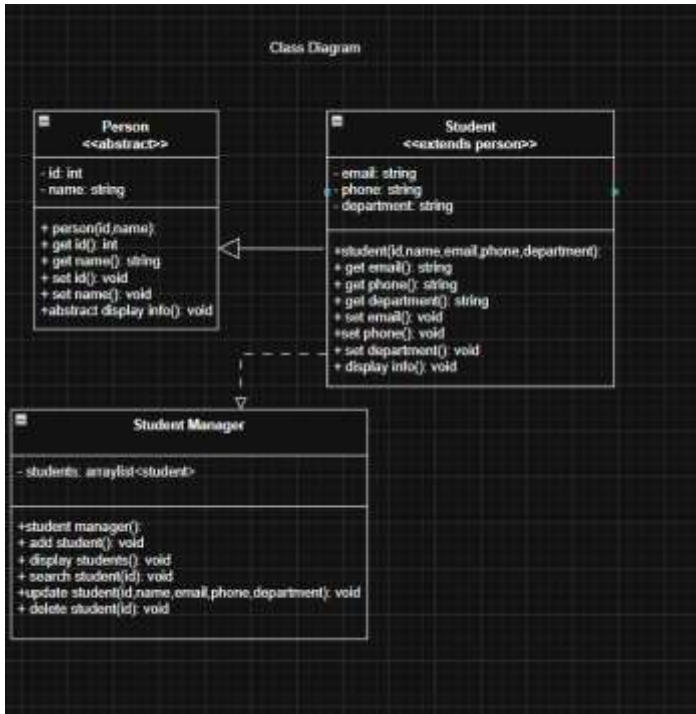
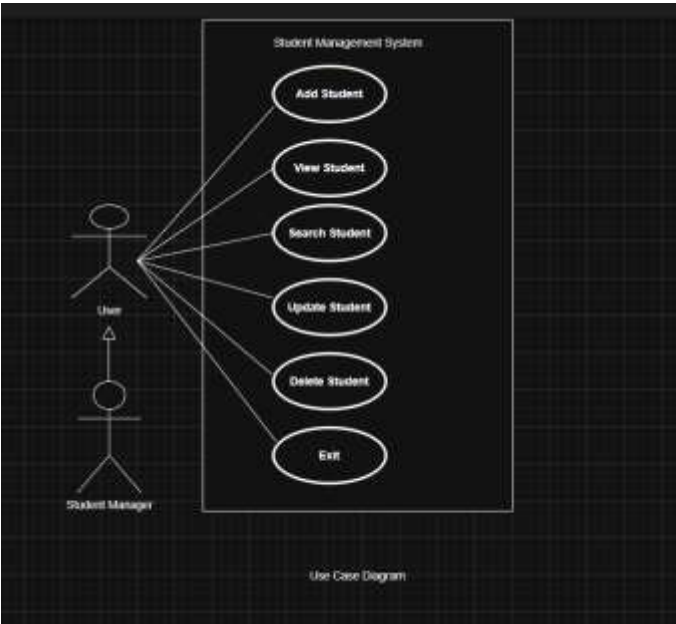
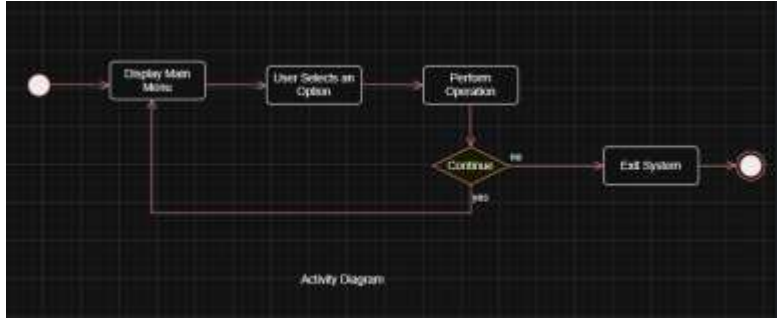
3.4 AI-Prompts Used and UML Diagrams

3.4.1.AI-Prompts Used

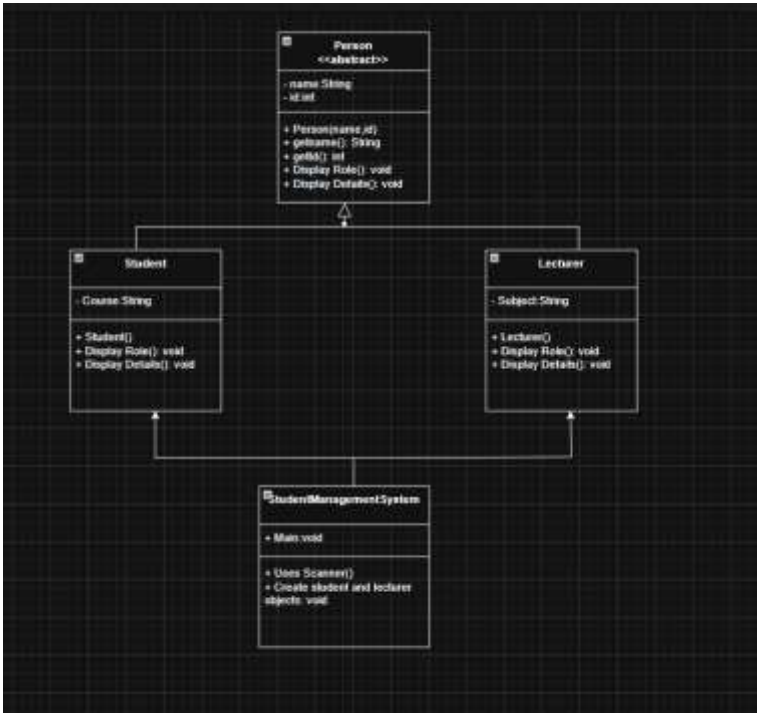
- “Give me a simple Java project that demonstrates Object-Oriented Programming concepts such as abstraction, inheritance, encapsulation, and polymorphism.”
- "Create a simple Java Student Management System. The program should include separate classes for Person, Student, StudentManager, and Main, and should be easy to understand."
- "Implement CRUD operations for the Student Management System using an ArrayList. Keep the implementation simple without file handling or database connectivity."
- "Simplify the Student Management System by removing file handling and custom exception handling. Don't want that much complexity."
- "Generate UML diagrams (Use Case Diagram, Class Diagram, and Activity Diagram) for the Student Management System based on the implemented Java classes."

3.4.2.UML Diagrams





Class Diagram(AI-Assisted Solution)



Class Diagram(Traditional Solution)

4.Task 3 – Critical Analysis

4.1 How AI Changed My Coding Approach

During this project, I developed the Student Management System using both the traditional programming approach and the AI-assisted programming approach. In the traditional implementation, I had to think about the program structure, write the code manually, and spend more time fixing errors.

When I used AI assistance, the development process became much faster. AI helped me generate code, organize the program into separate classes, and suggest better ways to implement different features. This allowed me to focus more on understanding the program instead of spending too much time writing repetitive code.

Although AI made the development easier, I still had to review, modify, and test the generated code before using it. Therefore, AI acted as a helpful assistant rather than replacing my own programming skills.

4.2 Did AI Improve or Reduce My Understanding?

In my experience, AI improved my understanding of Java programming. It explained object-oriented programming concepts such as abstraction, inheritance, encapsulation, and polymorphism in a simple way. It also helped me understand how to organize a program into different classes.

However, I also realized that depending completely on AI is not a good idea. If I simply copied the generated code without understanding it, I would not improve my programming skills. Therefore, I compared the AI-generated implementation with my traditional implementation and tested each part of the program myself.

Overall, AI made learning easier, but understanding the code was still my own responsibility.

4.3 Situations Where AI Should Not Be Used

Although AI is very useful, there are situations where it should not be used without careful consideration.

Students should not rely completely on AI when learning programming because writing code manually helps develop logical thinking and problem-solving skills.

AI should also not be trusted without verification when developing important systems such as banking, healthcare, or security applications. Since AI can sometimes generate incorrect or insecure code, developers must carefully review and test the output.

In addition, students should not submit AI-generated code directly without understanding it. They should always make sure the final work reflects their own knowledge and follows academic integrity guidelines.

4.4 Ethical Considerations in AI-Generated Code

Using AI in programming also raises ethical responsibilities. Developers should use AI honestly and acknowledge its assistance whenever required.

AI-generated code should always be reviewed and tested before being used because the programmer is responsible for the final software. Developers should also avoid copying AI-generated content without understanding it.

For this project, AI was used only as a supporting tool. I reviewed, modified, and tested all generated code before including it in the final implementation. This helped me learn the concepts while maintaining academic honesty.

5.Task 4 – Conclusion

This assignment helped me understand the differences between traditional programming and AI-assisted programming by developing two versions of a Student Management System.

The traditional implementation improved my coding skills because I had to write the program manually and apply object-oriented programming concepts on my own. On the other hand, the AI-assisted implementation helped me complete the project more quickly by providing coding suggestions, improving the program structure, and helping with debugging.

After comparing both implementations, I believe AI is a valuable tool for programmers. It can save time, improve productivity, and support learning. However, AI should not replace a programmer's own knowledge and critical thinking. Developers should always understand, review, and test AI-generated code before using it.

In conclusion, I believe AI will continue to play an important role in both programming education and the software industry. When used responsibly, it can help developers create better software while still encouraging continuous learning and improvement.

6. References

- [GeeksforGeeks Java Programming](#)

GeeksforGeeks. (2025). *Java Programming Language*.

- [Oracle - Object-Oriented Programming Concepts](#)

Oracle. (2024). *Object-Oriented Programming Concepts*.

- [Visual Paradigm UML Guide](#)

Visual Paradigm. (2025). *UML Diagram Guide*.

7.Git Hub Repository

- <https://github.com/thevindugimantha-blip/student-management-system.git>

